

N 皇后固定张量网络的 精确 TTN 收缩与 GPU 集群路线

底层代数 · 收缩不变量 · 对称表示 · 外存归并 · V100 实测 / A100 部署

文档版本	2026-07-30
数学路线	固定局部因子网络上的精确整数收缩
当前验证	Q(1) 至 Q(12) ; N=15 推进至 111/120 概念节点
GPU 验证	8×Tesla V100-SXM2 32 GB ; A100 部署方案已整理

技术路线整理稿

目录

阅读说明	5
N 皇后固定张量网络的精确 TTN 收缩技术路线	6
0. 结论先行	6
1. 两种“TTN 路线”必须区分	6
2. 从 N 皇后计数到局部因子网络	7
2.1 变量	7
2.2 为什么是固定局部网络	7
3. COPY 网络与精确吸收	8
3.1 普通张量网络形式	8
3.2 吸收恒等式	8
4. 二叉收缩树的底层不变量	9
4.1 子树的数学含义	9
4.2 合并规则	9
4.3 正确性归纳	9
5. 这与枚举、动态规划的边界	9
6. 收缩树规划：只优化拓扑	10
6.1 当前贪心评分	10
6.2 为什么最大 rank 不是全部	10
6.3 宽度的理论限制	11
7. 紧凑稀疏数组表示	11
7.1 坐标编码	11
7.2 等值连接	11
7.3 精确整数安全界	12
8. 对称性的统一理论	12
8.1 列反射：全局不变扇区	12
8.2 列反射商空间中的精确收缩	13
8.3 行反射：网络自同构与镜像 DAG	13
8.4 行反射基本域	13
8.5 第二层行反射表示块	14
9. 外存分块与有序 run 精确归并	14
9.1 原始内存瓶颈	14
9.2 分块贡献生成	14
9.3 sorted-runs	15
9.4 复杂度	15
10. 推荐执行架构	15
11. 实验结果与它们能说明什么	16
11.1 N=10 后端演进	16
11.2 N=11 校准	16

11.3 第二层行反射块的负结果	17
12. N=15 实验的准确解释	17
12.1 纯拓扑计划	17
12.2 内存安全预算实验	17
12.3 外存跨过 3.327 亿连接	18
12.4 “111/120” 不是什么	18
12.5 当前结论	18
13. 完整正确性链条	18
引理 1：局部因子计数	18
引理 2：COPY 吸收	19
引理 3：二叉收缩不变量	19
引理 4：列反射轨道商	19
引理 5：行镜像复用	19
引理 6：稳定子树的第二层行商	19
引理 7：外存归并	19
14. 已排除或暂不采用的方向	19
14.1 基于完整解的兼容矩阵秩构造	19
14.2 配置状态或 mask TTN	19
14.3 强制整棵树镜像	19
14.4 行基本域与列偶块的直接叠加	19
14.5 强制第二层行反射块	19
14.6 只提高一次性内存预算	19
15. 下一步最有理论价值的优化	20
15.1 纯拓扑收缩树搜索	20
15.2 共享键索引和预检的外存化	20
15.3 局部反射电荷基	20
15.4 模素数与 CRT	20
15.5 分布式有序 run	21
16. 推荐研究顺序	21
17. 实现与复现入口	21
18. 最终理论判断	22
精确 TTN 的 GPU 集群迁移路线	23
当前已实现的阶段	23
为什么不能直接调用 GEMM	23
使用	24
为什么仍然精确	25
当前多 GPU 算法	25
8×V100 实测	25
单 GPU 数据通路	25
完整多 GPU / 多节点 join 分解	26

集群存储要求	26
后续实施顺序	26
V100 与 A100 的部署建议	27
附录：复现入口与验证边界	28

阅读说明

本文将两部分内容整合为一个连续技术说明：第一篇解释精确 TTN 计数路线的数学基础、正确性与实验边界；第二篇给出面向 V100/A100 服务器的 GPU 和多卡迁移架构。文中的 TTN 主要指局部因子网络的二叉收缩树，而不是由完整解枚举后压缩得到的状态 TTN。

层次	核心职责	精确性依据
局部网络	用 W_{ij} 表示列与对角线兼容性	0/1 因子乘积就是合法性指示值
收缩树	选择有限和的结合顺序	部分收缩不变量与整数分配律
对称表示	列反射轨道商、行反射 DAG 复用	群作用下的张量等价
外存归并	分块生成并精确合并同键贡献	每个连接贡献恰处理一次
GPU 多卡	互斥键范围并行 sort/reduce	相同键只进入一张卡

N 皇后固定张量网络的精确 TTN 收缩技术路线

0. 结论先行

这条路线把 N 皇后计数写成一个固定的、只含局部 0/1 因子的闭合张量网络，然后选择一棵二叉收缩树，在非负整数半环上做完全精确的乘法与求和。

它已经独立得到

$$Q(1), \dots, Q(11) = 1, 0, 0, 2, 10, 4, 40, 92, 352, 724, 2680.$$

这里的“TTN”主要是张量网络的二叉收缩树，不是先枚举所有解、再把解指示函数压成一个低键维 TTN。当前默认路线由五层组成：

1. 用固定二体兼容张量表示所有列冲突和对角线冲突；
2. 用恒等式精确吸收 COPY 张量；
3. 用只读取网络拓扑的规划器构造二叉收缩树；
4. 用列反射偶表示和行反射镜像 DAG 消除严格重复；
5. 用排序紧凑数组和外存有序 run 归并执行超大稀疏收缩。

整条路线没有：

- 枚举完整 N 皇后解；
- 沿棋盘行递归生成合法部分棋盘；
- 使用列、主对角线、副对角线 bitmask；
- 建立以攻击掩码或部分配置为键的动态规划表；
- 使用 SVD 截断、浮点阈值或近似张量。

但必须准确说明：精确稀疏张量收缩仍会保存和处理中间张量的非零边界坐标。这些坐标是固定张量网络的基坐标，不是完整解列表，也不是攻击掩码 DP 状态。如果连中间张量坐标都不允许出现，那么任何可执行的基底形式精确张量收缩也会被排除。

当前 $N=15$ 尚未在这台 16 GiB 机器上得到最终标量。最好的受控实验完成了 120 个概念收缩节点中的 111 个，精确跨过了 332,706,850 对连接，下一步需要 1,150,580,209 对连接。这个结果定位了资源瓶颈，但不是 $Q(15)$ 的部分值，也不能按 111/120 理解为完成了 92.5% 的算术工作。

1. 两种“TTN 路线”必须区分

最初技术文档研究的是兼容矩阵秩。定义完整合法性指示函数

$$F_N(c_0, \dots, c_{(N-1)}) \in \{0, 1\},$$

对行划分 $A|B$ 构造

$$M_{(A|B)}(a, b) = F_N(a, b).$$

矩阵秩给出跨该切口的 MPS/TTN 精确键维下界。这是一条研究“完整解指示态是否存在低键维表示”的路线。原方案通过枚举完整解来构造稀疏兼容矩阵，因此在后来加入“不得使用精确枚举或配置状态动态规划”的约束后，它只能作为历史上的结构诊断，不能再作为当前求解器的计算基础。

当前路线研究的是另一个对象：

$$Q(N)=\text{Contract}(\text{固定局部约束网络}).$$

它不先构造完整的 F_N ，也不对由完整解得到的矩阵做秩分解。它直接收缩 局部约束因子。二者的差异是：

问题	兼容矩阵秩路线	当前精确收缩路线
基本对象	完整合法性函数 F_N	固定局部因子 $W_{(ij)}$
TTN 含义	F_N 的状态表示	局部网络的二叉收缩树
是否先需要完整解支持	原实现需要	不需要
主要理论量	切分矩阵秩、键维	边界秩、nnz、连接基数
当前用途	历史诊断	实际精确计数

因此，当前技术路线不是“秩扫描成功后构造出的低秩 TTN”，而是从局部因子图 直接得到的一棵精确 contraction TTN。

本文后续实验表中的 **rank** 若无特别说明，指中间张量的自由指标个数（tensor order），不是兼容矩阵的线性代数秩，也不是已经证明的 Schmidt rank。二者都可影响 TTN，但不是同一个量。

MPS/MERA 目前也不在计数主路径上。MPS 的切口秩和 MERA 的因果切口仍可用于 表示能力研究；但在不依赖完整解矩阵、又没有找到严格降低后续宽度的跨块可逆 变换之前，直接构造 MERA 不会自动改善式 (1) 的收缩。当前研究顺序已经从“rank-first → TTN → MERA”收敛为“固定局部网络 → 精确 contraction TTN → 只在有严格块结构证据时再考虑 MERA”。

2. 从 N 皇后计数到局部因子网络

2.1 变量

每一行恰放一个皇后。令

$$c_i \in \{0, 1, \dots, N-1\}$$

表示第 i 行皇后的列坐标。这样“每行一个皇后”已经由变量定义保证。

对两行 $i < j$ ，定义距离

$$d_{(ij)} = j - i$$

以及二体兼容因子

$$W_{(ij)}(a, b) = 1[a \neq b] 1[|a - b| \neq d_{(ij)}].$$

第一个指示函数排除同列，第二个排除同一对角线。

于是

$$(Q(N) = \sum_{(c_0, \dots, c_{(N-1)})=0}^{(N-1)} \prod_{(0 \leq i < j < N)} W_{(ij)}(c_i, c_j)) \quad (1)$$

因为对任意完整列赋值 c ：

- 若所有皇后互不攻击，乘积中每个因子都是 1，该赋值贡献 1；
- 若至少有一对冲突，对应因子为 0，整个乘积贡献 0。

式 (1) 是整条路线的根。最终程序只是在改变这个有限和的结合顺序，而没有 改变被求和的函数。

2.2 为什么是固定局部网络

网络的非平凡叶张量只由 N 和行距 $j-i$ 决定：

- 每行一个恒等的一阶张量

$$\text{ONE}_i(c_i)=1;$$

- 每对行一个二阶张量 $W_{(ij)}$;
- 一个值为 1 的零阶标量叶，用于统一二叉计划。

实现中的叶数为

$$L=N+C(N,2)+1,$$

所以完整二叉树有

$$L-1=N+C(N,2)=\frac{N(N+1)}{2}$$

个概念收缩节点。 $N=15$ 时有 121 个叶和 120 个概念节点。

这些张量在收缩开始前一次性按公式构造。规划器不会根据某一步碰到的棋盘 配置临时生成新约束。

3. COPY 网络与精确吸收

3.1 普通张量网络形式

在普通张量网络中，一条边通常只连接两个张量；但同一个 c_i 同时出现在 $N-1$ 个二体因子中。可以为第 i 行复制 $N-1$ 个端点，并放置 COPY 张量

$$\Delta_i(x_1, \dots, x_{(N-1)}) = (1, x_1 = \dots = x_{(N-1)}, ; 0, \text{ 否则. })$$

闭合网络为

$$Z_N = \sum_{\text{所有网络指标}} \prod_i \Delta_i \prod_{(i < j)} W_{(ij)}.$$

COPY 强制同一行在全部 pair 因子上使用相同列坐标，因此

$$Z_N = Q(N).$$

3.2 吸收恒等式

显式 COPY 张量会产生很多临时重复指标。利用

$$\sum_{\alpha} (x_1, \dots, x_k) \Delta(x_1, \dots, x_k) \prod_{\alpha=1}^k F_{\alpha}(x_{\alpha}) = \sum_{(c=0)}^{(N-1)} \prod_{\alpha=1}^k F_{\alpha}(c), \quad (2)$$

可以把 COPY 的所有端点严格识别成一个共享超指标 c_i 。

式 (2) 只是 Kronecker delta 的定义，不涉及：

- 截断；
- SVD；
- 近似低秩分解；
- 对合法配置的搜索；
- 对中间结果的记忆化。

吸收后得到式 (1) 的因子超图。实验上， $N=7$ 的显式 COPY 网络最大中间 rank 为 12、最大 nnz 为 12,223,650；吸收后分别降到 5 和 4,540，而最终标量都为 40。这说明改善来自精确网络恒等式，而不是改变问题。

4. 二叉收缩树的底层不变量

4.1 子树的数学含义

设全部叶因子集合为 F 。收缩树的一个节点对应叶子子集 $S \subseteq F$ 。

令 $B(S)$ 是同时出现在 S 内因子和 $F \setminus S$ 外部因子 中的行变量集合，称为该子树的边界。只在 S 中出现的变量称为内部变量 $I(S)$ 。

该节点所表示的精确中间张量定义为

$$T_S(c_{(B(S))}) = \sum_{(I(S))} \prod_{(f \in S)} f(c_{(scope(f))}). \quad (3)$$

这句话是整个收缩器最重要的不变量：

中间张量是某一固定叶因子集合的精确部分收缩；它保留且只保留仍与网络外部 相连的边界指标。

它不是“已经摆完前 k 行之后的搜索状态”。子树可以包含任意行约束，也不服从从上到下的棋盘生成顺序。

4.2 合并规则

现有两个活动张量 T_A, T_B ，对应互不重叠的叶集合 A, B 。令

$$S = A \cup B.$$

合并时：

1. 两张量共有的同名指标必须取相同坐标；
2. 仍在 S 外出现的指标保留为 $B(S)$ ；
3. 已不在外部出现的指标立即精确求和；
4. 对匹配项做非负整数乘法，并按相同输出坐标求和。

若用 x 表示保留边界、 y 表示这一步消去的指标，则抽象地写成

$$T_S(x) = \sum_y T_A(x_A, y_A) T_B(x_B, y_B), \quad (4)$$

共有标签的坐标在乘法前被等值连接。

4.3 正确性归纳

叶节点。 每个叶张量按定义等于对应局部因子，因此满足式 (3)。

归纳步。 假设 T_A, T_B 都满足式 (3)。它们的内部叶集合不重叠，将两式相乘后，所有内部求和变量彼此独立；再对不再连接外部的共有或单侧指标 求和，恰好得到 $A \cup B$ 的式 (3)。这就是式 (4)。

根节点。 根的叶集合是 F ，外部为空，所以 $B(F) = \emptyset$ 。式 (3) 退化为零阶张量

$$T_-(F) = Q(N).$$

因此任意合法的二叉结合顺序都会得到同一个精确标量。规划器影响时间和空间，不影响答案。

5. 这与枚举、动态规划的边界

从纯代数角度看，张量收缩和 sum-product 变量消去都依赖分配律。不能声称它们在抽象代数上毫无关系。但当前实现与被禁止的方法有清晰的对象差异：

方法	被保存的对象	转移或操作
完整解枚举	每个合法完整棋盘	逐解产生、计数

方法	被保存的对象	转移或操作
部分棋盘搜索	已放若干行的合法布局	尝试下一列并剪枝
bitmask DP	列/对角线占用掩码及计数	按行递推、合并相同掩码
当前收缩	固定因子子集的边界张量系数	张量等值连接、乘法、求和

当前规划器也不读取：

- 张量值；
- 非零坐标模式；
- 某个部分棋盘是否合法；
- 已知 $Q(N)$ ；
- 搜索分支的成功率。

它只读取指标关联、指标维数、叶因子名称及由名称确定的网络对称轨道。

最准确的表述是：

当前方法是固定因子网络上的精确代数收缩；它不是解生成器，也不是以配置或攻击掩码为状态的动态规划。稀疏中间张量会显式存储非零边界基坐标，这是精确张量表示本身，而不能伪称为“完全不接触任何配置坐标”。

6. 收缩树规划：只优化拓扑

6.1 当前贪心评分

对每一对候选活动张量，先根据网络外部标签计算合并后的边界和维数。默认 **coverage-first** 评分按以下优先级最小化：

1. 是否无共享指标；优先避免外积；
 2. 输出稠密包络
- $$D_{\text{out}} = \prod_{(\ell \in B(S))} \dim(\ell);$$
3. 输出 rank $|B(S)|$ ；
 4. 在上述成本相同时，优先让子树覆盖更多 PAIR 约束；
 5. 再优先可直接镜像复用或可形成镜像配对的候选；
 6. 最后用确定性的活动数组位置打破平局。

N 皇后中每个列指标维数都是 N，所以 rank 为 r 的稠密包络为

$$N^r.$$

“覆盖更多 pair 因子”是一种纯拓扑代理：同样的自由边界下，内部已经包含更多固定约束通常会让后续实际张量更稀疏。规划器没有查看这种稀疏性是否真的发生。

6.2 为什么最大 rank 不是全部

两个候选树可能有相同最大 rank，却有完全不同的非零数和等值连接大小。实际瓶颈至少由三个量共同决定：

boundary rank, nnz(T), J=匹配连接对数.

第二层行反射实验就是反例：强制对称优先仍保持 $N=15$ 计划最大 rank 13，但约束进入子树的次序变差，稀疏性损失大于对称商带来的二倍压缩。

因此后续树搜索应使用拓扑可计算的多目标代理，而不能只盯最大 rank；同时，为了遵守当前边界，搜索阶段仍不能偷看中间数值或合法配置。

6.3 宽度的理论限制

COPY 吸收后的原始变量图在每对行之间都有一个 $W_{(ij)}$ ，其 primal graph 是完全图 K_N 。因此一般的稠密变量消去宽度随 N 线性增长，最坏存储随 N 指数增长。稀疏 0/1 结构和对称性可以大幅改变常数与实际 nnz，但不会自动把完全图的宽度变成常数。

这解释了当前路线的理论定位：

- 它是严格可行的精确计数方法；
- 它并没有得到多项式复杂度证明；
- 对称性通常提供常数因子或复用因子；
- 真正决定 $N=15$ 后段困难的是收缩树边界与稀疏连接爆炸。

7. 紧凑稀疏数组表示

7.1 坐标编码

对 rank r 的张量，固定标签顺序 $(\ell_1, \dots, \ell_r)$ ，坐标为 $(x_1, \dots, x_r)$ 。使用混合进制扁平键

$$k = (((x_1 d_2 + x_2) d_3 + x_3) \cdots) d_r + x_r, \quad (5)$$

其中 $d_t = \dim(\ell_t)$ 。本问题中通常 $d_t = N$ 。

每个非零项用两个连续的 uint64 数组元素保存：

- `keys[p]`：严格递增的扁平坐标；
- `values[p]`：该坐标的精确非负整数系数。

持久张量的逻辑存储为每个非零项 16 字节，不保存稠密零。

7.2 等值连接

合并 A, B 时，把所有共有标签的坐标分别重新编码为共享键。对共享键排序并分组。若某个共享键组在左右两侧分别有 $n_A(g)$ 和 $n_B(g)$ 行，则该组产生

$$n_A(g)n_B(g)$$

个精确乘法贡献。总连接基数为

$$J = \sum_g n_A(g)n_B(g). \quad (6)$$

每一对匹配行生成一个输出键和系数乘积；输出键排序后，相同键用 uint64 加法归并。

因此稀疏收缩的成本不只取决于输入 nnz：

$$S_A = \text{nnz}(A), S_B = \text{nnz}(B), J, S_{\text{(out)}}.$$

在坏情况下 J 可以远大于两个输入，也可以远大于最终去重后的 $S_{\text{(out)}}$ 。

7.3 精确整数安全界

叶因子是 0/1。任一中间张量元素都等于某些原始列赋值贡献的计数，因此

$$0 \leq T_S(x) \leq N^N. \quad (7)$$

当前代码只在

$$N^N < 2^{64}$$

时使用 uint64 后端。对 $N=15$ ，

$$15^{15} = 437,893,890,380,859,375 < 2^{64} = 18,446,744,073,709,551,616.$$

所以当前范围内的乘加是精确整数运算，不是模 2^{64} 的有意计算。若使用带权的行反射基本域，界相应检查为 $2N^N < 2^{64}$ 。

超出安全界后，合理扩展是使用多个素数独立收缩，并在乘积模数足够覆盖严格上界后用中国剩余定理恢复整数，而不是允许静默溢出。

8. 对称性的统一理论

对称性必须按它作用在什么对象上来分类。当前路线实际用了三种不同机制：

1. 不变张量的轨道商表示：列反射偶块；
2. 同构子问题的计算复用：行反射镜像 DAG；
3. 最终配置轨道的基本域加权：行反射基本域。

它们不能混为“把解数除以 2”。

8.1 列反射：全局不变扇区

定义列反射

$$C(c_1, \dots, c_r) = (N-1-c_1, \dots, N-1-c_r).$$

对任意 pair 因子，

$$W_{(ij)}(a, b) = W_{(ij)}(N-1-a, N-1-b),$$

因为“不相等”和绝对列差都保持不变。ONE 叶也显然不变。

若两个张量分别满足

$$A(x) = A(Cx), \quad B(y) = B(Cy),$$

则它们的乘积、共有指标匹配和内部指标求和仍满足同样的不变性。因此由归纳可知收缩树上的每个中间张量都满足

$$T_S(x) = T_S(Cx). \quad (8)$$

所以只需为每个二元轨道保存一个规范代表

$$\text{can_C}(k) = \min(k, Ck). \quad (9)$$

这可理解为使用未归一化偶轨道基

$$|k \square_+ \rangle = |k \square \rangle + |Ck \square \rangle.$$

更精确地说，实现保存的是不变函数在商空间上的系数，不做带平方根归一化的量子态基变换。

若 N 为偶数，任何非标量坐标都没有 C 固定点，所以坐标轨道严格成对。若 N 为奇数，所有自由坐标同时位于中心列 $(N-1)/2$ 时存在唯一可能的固定坐标，必须单独处理。

8.2 列反射商空间中的精确收缩

不能简单把两个操作数都只取一半后直接连接，因为它们各自选择的规范方向可能不一致。当前算法：

1. 选择较大的操作数作为规范锚点；
2. 保持锚点在规范方向；
3. 把较小操作数的非固定轨道展开为原方向和 C 方向；
4. 在完整的共享坐标上做等值连接；
5. 将输出重新规范化为 $\min(k, C_k)$ ；
6. 若输出坐标是固定点、但被消去前的联合赋值不是固定点，则乘精确轨道权 2。

最后一步的原因是：一个二元联合轨道可能在消去内部指标后塌缩到同一个固定输出坐标。若不乘 2，就会漏掉轨道的另一个成员。

锚定一侧、只展开较小一侧是一种执行优化，不改变群轨道计数。

8.3 行反射：网络自同构与镜像 DAG

定义行反射

$$H:i \mapsto N-1-i.$$

它作用在叶和标签上：

$$ONE_i \mapsto ONE_{(N-1-i)},$$

$$PAIR_{(i,j)} \mapsto PAIR_{(N-1-j, N-1-i)},$$

$$column_i \mapsto column_{(N-1-i)}.$$

行距保持不变，所以这是因子网络的自同构。

若子树 S' 的叶集合恰为 $H(S)$ ，则

$$T_{(S')}(Hx) = T_S(x). \quad (10)$$

因此，只要一个活动子树的镜像已经算过，另一个子树可通过标签重命名和必要的坐标轴置换复用其 **keys** 与 **values**，不再执行整数乘加。概念树仍覆盖全部叶，实际执行图则成为带 **mirror_reuse** 节点的 DAG。

这不是把某个任意中间张量减半。一般情况下 $H(S) \neq S$ ，行反射把当前子树映到另一棵子树，而不是把它映回自身。所以行反射在默认树上主要提供 **跨子树复用**，不是局部表示商。

列反射 C 改变坐标值，行反射 H 重排指标标签，二者交换：

$$CH = HC.$$

因此镜像 DAG 可以与列反射偶表示直接叠加。

8.4 行反射基本域

也可以不做 DAG，而在最终合法棋盘的 H 轨道中选一个代表。选镜像行 $i \leq j = N-1-i$ 。合法棋盘必有 $c_i \neq c_j$ ，所以每个合法行反射轨道恰有一个成员满足 $c_i < c_j$ 。把原 **pair** 叶替换为

$$W'_{(ij)}(a,b) = 2W_{(ij)}(a,b)1[a < b] \quad (11)$$

即可一次性计入整个二元轨道。

这也是精确的固定局部变换，但它与列反射商不能直接叠加，因为

$$c_i < c_j \rightarrow N-1-c_i > N-1-c_j.$$

即基本域条件破坏了 C 不变性。若要同时使用，需要处理完整 $Z_2 \times Z_2$ 群及其固定扇区，不能简单连续做两次除二。

8.5 第二层行反射表示块

如果某个子树同时满足

$$H(S)=S, H(B(S))=B(S), (12)$$

那么它的中间张量自身对行标签反射不变。由于 H 与 C 交换，在列反射商空间中可定义诱导对合

$$H(k)=\min(Hk, CHk). (13)$$

于是可进一步只存

$$\min(k, H(k))$$

的代表。若下一步收缩不再保持该稳定子群，则先精确展开另一方向。

该表示在数学上正确，且对单个自镜像张量近似再减半。但默认 $N=15$ 树没有非标量自镜像中间节点；为了制造它们而使用 `symmetry-first` 规划会延迟一些重要约束的合并，产生更稠密的中间张量。

这给出一个一般原则：

只有当对称稳定子群自然地出现在低宽度、高约束覆盖的子树上时，局部对称商才可能有净收益。为得到一个二倍表示压缩而强制改变整棵树，可能付出更大的稀疏性代价。

9. 外存分块与有序 run 精确归并

9.1 原始内存瓶颈

普通稀疏等值连接会一次性物化式 (6) 的全部 J 对左右行索引、输出键、输出值和排序副本。因此内存更接近 $O(J)$ ，而不是只由最终张量 S_{out} 决定。

`--max-join-pairs` 先精确计算 J ，若超过预算，就在大数组分配前停止。这是安全闸门，不是：

- 截断；
- 抽样；
- 近似；
- 返回部分计数。

9.2 分块贡献生成

设置 `--join-chunk-pairs=B` 后，匹配共享键组仍被完整遍历，但每次最多生成约 B 条贡献。每条贡献执行与内存版完全相同的：

$$(\text{left row}, \text{right row}) \mapsto (\text{canonical output key}, \text{exact product}).$$

分块只改变处理顺序。

9.3 sorted-runs

当前默认外存策略为：

1. 在内存中累计一个贡献块；
2. 按输出键排序；
3. 先对块内重复键精确求和；
4. 把严格递增的唯一键和值写成一个有序 run；
5. 用 C++ 缓冲优先队列对全部 run 做 k 路归并；
6. 归并过程中再次对跨 run 的相同键精确求和；
7. 最终得到磁盘映射的严格递增 keys 与 values。

设第 t 块对输出键 k 的贡献和为 $s_t(k)$ 。最终归并计算

$$\sum_t s_t(k) = \sum_t ((\text{全部匹配行对；输出键}=k)) v_{Av_B}. \quad (14)$$

因为全部数都是非负整数，并且已由式 (7) 保证不溢出，分块、块内归并和跨 run 归并只是有限整数加法的重结合，结果与一次性排序逐位相同。

9.4 复杂度

令块大小为 B ，run 数为 m 。粗略地：

- 贡献生成总量为 J ；
- 块内排序总成本约为

$$\sum_t O(B_t \log B_t);$$

- 多路归并比较成本约为

$$O(R \log m),$$

其中 R 是块内去重后的 run 记录总数；

- 持久磁盘量由 run 和最终 $S_{(out)}$ 决定，坏情况下仍为

$O(J)$ ；

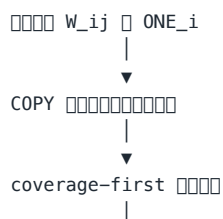
- 归并器工作内存约为块缓冲和各 run 的顺序读缓冲，而不需要一次性保存全部

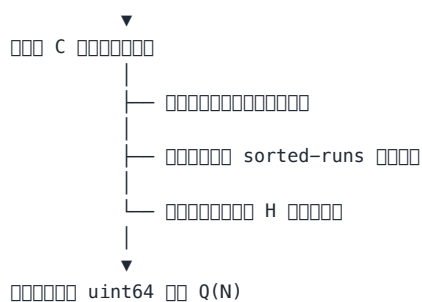
J 条贡献。

当前共享键排序和连接基数预检仍会触碰大输入索引及磁盘映射页，因此 RSS 峰值并未完全降到 $O(B)$ 。这正是下一阶段需要外存化的部分。

10. 推荐执行架构

当前推荐默认路线可以写成以下数据流：





这几个层次相互独立：

- 因子定义保证“算的是什么”；
- 收缩不变量保证“任意步骤为什么正确”；
- 规划器决定“以什么结合顺序算”；
- 对称表示决定“如何不重复存同一轨道”；
- DAG 决定“如何不重复算同构子树”；
- 外存归并决定“如何在有限 RAM 中执行同一个稀疏乘加”。

任何一层优化失败，都不应回退到完整枚举或配置状态 DP；可以保留相同数学网络，只替换表示或规划。

11. 实验结果与它们能说明什么

11.1 N=10 后端演进

后端	最大持久张量	非零乘加	峰值 RSS	时间
完整排序坐标	29.99 MB	3,847,131	324.9 MB	0.482 s
行反射基本域	18.23 MB	2,779,999	221.4 MB	0.348 s
列反射偶块	15.00 MB	1,923,566	221.4 MB	0.381 s
行镜像 DAG	8.95 MB	1,382,235	133.1 MB	0.313 s
行镜像 DAG + 列偶块	4.48 MB	691,118	93.1 MB	0.271 s

组合默认路线执行 44 次实际收缩、复用 11 个镜像节点，共 55 个概念节点；最大边界 rank 为 8，最大列偶轨道 nnz 为 279,741。

这些数据说明：

- 紧凑数组消除了 Python 字典和稠密零的主要浪费；
- 列反射商接近严格二倍压缩；
- 行反射 DAG 减少的是算术节点，不只是存储；
- 两种机制可乘性叠加，因为 C 与 H 交换。

11.2 N=11 校准

默认对称路线得到精确标量 2680：

指标	内存版	sorted-runs 外存版
实际/概念收缩	58/66	58/66
镜像复用	8	8

指标	内存版	sorted-runs 外存版
最大 rank	9	9
最大列偶 nnz	11,638,952	11,638,952
非零乘加	29,253,099	29,253,099
峰值 RSS	2.98 GB	1.52 GB
时间	5.616 s	7.327 s

外存版用时间换内存，但标量、树结构、rank、nnz 和乘加数都不变。

11.3 第二层行反射块的负结果

N	路线	最大列偶 nnz	最大实际存储 nnz	乘加	时间
10	默认树 + 列偶	279,741	279,741	691,118	0.271 s
10	对称优先树 + 双反射	2,500,642	1,251,332	5,418,681	1.217 s
11	默认树 + 外存 run	11,638,952	11,638,952	29,253,099	7.327 s
11	对称优先树 + 双反射	29,869,288	14,940,623	66,506,550	17.068 s

对单个自镜像张量，第二层商确实接近减半；但为了让子树自镜像，收缩树先生成了大 4-9 倍的列偶张量。这个实验验证了表示公式，却否定了当前强制树策略。

12. N=15 实验的准确解释

12.1 纯拓扑计划

N=15 有：

- 121 个叶；
- 120 个概念收缩节点；
- 20 个行反射镜像复用节点；
- 最大计划边界 rank 13；
- 最大稠密包络

$$15^{(13)}=1,946,195,068,359,375.$$

稠密包络不是实际分配量，但表明后段边界的理论规模极大。

12.2 内存安全预算实验

最大允许连接对	完成节点	下一连接对	峰值 RSS
5,000,000	107/120	7,837,193	0.70 GB
20,000,000	108/120	46,365,011	1.65 GB
50,000,000	109/120	66,670,618	5.29 GB
70,000,000	110/120	332,706,850	4.98 GB

这些“完成节点”包含实际收缩和镜像复用。预算停止发生在大数组分配之前，已完成张量都是精确的，但没有完整根标量。

12.3 外存跨过 3.327 亿连接

sorted-runs 完整处理了

$$J=332,706,850$$

个连接贡献，生成：

- rank 9 的列偶张量；
- 332,706,850 个非零列偶轨道；
- 单张量逻辑数组大小 5.323 GB；
- 活动唯一数组逻辑大小 7.225 GB；
- 91 次实际收缩；
- 20 次镜像复用；
- 共推进到 111/120。

可信输出记录的本轮时间为 88.497 s，峰值 RSS 为 7.04 GB。下一节点的精确 预检为

$$J_{\text{(next)}}=1,150,580,209.$$

12.4 “111/120” 不是什么

它表示二叉计划的 111 个结构节点已经完成，不表示：

- 已经得到 $Q(15)$ 的 111/120；
- 已经完成 92.5% 的乘加；
- 剩余时间只有前面时间的 9/111；
- 可以从当前张量推断最终标量。

后 9 个节点包含 rank 12、rank 13 边界，单个节点的连接量可能超过前面全部 节点之和。精确标量只在 120 个节点全部完成后出现。

12.5 当前结论

当前本机结果支持：

精确 TTN 收缩在数学上可以解这个问题，并已实际解到 $N=11$ ；当前默认 实现和 16 GiB 本机尚不能完成 $N=15$ 。

它不支持：

任何精确 TTN 都不可能完成 $N=15$ 。

尚未排除更好的收缩树、不同的精确对称基、更彻底的外存索引或更大的外部资源。

13. 完整正确性链条

可以把整条路线形式化为七个引理。

引理 1：局部因子计数

式 (1) 中每个完整赋值的乘积等于其 N 皇后合法性指示值，因此总和为 $Q(N)$ 。

引理 2：COPY 吸收

式 (2) 对每一行成立，所以显式 COPY 网络与共享超指标网络有相同闭合标量。

引理 3：二叉收缩不变量

收缩树每个节点都满足式 (3)。由叶节点和式 (4) 归纳可证。

引理 4：列反射轨道商

所有叶在 C 下不变，收缩保持不变性；规范代表、固定点和轨道权处理恢复完整收缩，因此商空间收缩与完整坐标收缩相同。

引理 5：行镜像复用

网络自同构给出式 (10)。对镜像叶集合做标签重命名得到完全相同的张量系数，所以复用节点与重新收缩等价。

引理 6：稳定子树的第二层行商

当且仅当式 (12) 保证局部稳定时，式 (13) 在列商上定义良好；保存轨道代表并在破坏稳定前精确展开，与完整列偶张量等价。

引理 7：外存归并

每个匹配行对恰被生成一次；块内和跨 run 归并按式 (14) 汇总全部同键贡献。因此分块大小和 run 数不改变输出张量。

由引理 1-7，默认后端最终根标量严格等于 $Q(N)$ 。

14. 已排除或暂不采用的方向

14.1 基于完整解的兼容矩阵秩构造

它可用于研究状态 TTN 的键维下界，但原实现依赖完整解投影，与当前限制不兼容，不作为计数路径。

14.2 配置状态或 mask TTN

任何把列和两类对角线占用情况当作键、按行推进的实现，本质上属于配置状态递推，不进入当前路线。

14.3 强制整棵树镜像

早期实验把 $N=10$ 最大 rank 从 8 推到 10。网络对称不等于最优收缩树必须在每层对称。

14.4 行基本域与列偶块的直接叠加

式 (11) 的不等式在列反射下翻转，破坏列偶不变性。不能顺序套用两个“减半”。

14.5 强制第二层行反射块

数学正确但实测净负收益，原因是收缩树的稀疏性损失大于局部二倍压缩。

14.6 只提高一次性内存预算

$N=15$ 的瓶颈已从千万级进入十亿级连接。单纯提高阈值会物化更多临时数组，不改变增长机制。

15. 下一步最有理论价值的优化

15.1 纯拓扑收缩树搜索

保持规划器 value-blind，扩大搜索空间：

- 随机化贪心；
- 局部树旋转；
- 子树交换；
- hypergraph partition；
- 以边界 rank、稠密包络、约束覆盖和镜像复用为多目标评分。

核心目标不是只把 rank 13 改成 12，而是让约束尽早进入子树，从而同时降低 实际 nnz 和式 (6) 的连接基数。

为了保持方法边界，候选树选择阶段不应运行完整中间张量来“试一下谁更稀疏”；只能使用网络拓扑可计算的代理。

15.2 共享键索引和预检的外存化

当前输出贡献已经可以流式，但大磁盘张量参与下一步时，共享键编码、排序和计数仍造成文件页驻留。下一步应：

- 分块生成共享键索引；
- 写成按共享键排序的磁盘 run；
- 流式合并左右共享键组；
- 在组级直接累计 $n_A(g)n_B(g)$ ；
- 同一遍扫描驱动后续贡献块生成。

这不会降低总 J，但可以把预检和索引阶段的 RAM 从“与大输入索引同阶”压到“与块和读缓冲同阶”。

15.3 局部反射电荷基

当前列反射 C 是同时作用于全部自由指标的全局轨道商。更深入的表示是把每个单指标空间按局部列反射

$$R|c\rangle = |N-1-c\rangle$$

分解为偶、奇电荷：

$$|c\rangle_{\pm} = |c \pm |N-1-c\rangle.$$

二体因子 W_{ij} 与 $R \otimes R$ 对易，因此只连接总反射电荷相容的块。若重新引入适当的 COPY/融合张量，COPY 会表现为 Z_2 电荷守恒。这可能把大张量改写成块稀疏电荷网络。

但这里有一个重要警告：

精确基变换本身不减少总向量空间维数。只有电荷选择规则让大量块严格为零，并且块结构与收缩树相容时，才会有净收益。

因此应先在 $N=8\sim 11$ 做完全精确的块尺寸与连接量基准，再用于 $N=15$ 。

15.4 模素数与 CRT

当整数界超过 uint64 时，在多个机器字素数域上执行完全相同的张量收缩。若模数乘积 M 满足

$$M > N^N$$

或满足所采用对称权的更强上界，就可唯一恢复非负整数结果。这是精确算术扩展，不是有限域秩扫描，也不是概率性答案。

15.5 分布式有序 run

式 (14) 的 run 天然可以独立生成，再做层次化精确归并。若允许多盘或多机，可以把超大连接的主要压力转化为顺序 I/O 和归并带宽。它解决资源分布，不改变指数宽度。

16. 推荐研究顺序

按理论收益与实现风险，建议：

1. 固化当前默认路线及 $N \leq 11$ 回归测试；
2. 外存化共享键索引和连接预检；
3. 实现纯拓扑随机树搜索与局部旋转；
4. 用 $N=10, 11$ 的 nnz 和 J 作为结果评估，但不把数值反馈进默认 value-blind 规划器；
5. 原型化局部 Z_2 电荷块；
6. 只有小尺寸块结构有净收益时再重新挑战 $N=15$ ；
7. 若仍无法完成，再考虑多盘、多机和 CRT。

判断一次优化是否值得保留，应同时看：

$\max r, \max nnz, \max J, \text{峰值 RSS, 磁盘量, 总时间}.$

不能只根据“对称性多了一层”或“最大 rank 没变”判断。

17. 实现与复现入口

推荐入口：

```
/Users/susstring/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/
python3 \
  python/contract_symmetric_parity_ttn.py \
  --n-min 1 --n-max 11
```

$N=15$ 当前已验证的受控边界：

```
/Users/susstring/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/
python3 \
  python/contract_symmetric_parity_ttn.py \
  --n-min 15 --n-max 15 \
  --join-chunk-pairs 1000000 \
  --streaming-merge-strategy sorted-runs \
  --max-join-pairs 332706850 \
  --summary \
  results/algebraic_symmetric_parity_ttn/summary_n15_sorted_runs1m_trusted_
budget332706850.jsonl
```

关键代码：

- algebraic_ttn/packed.py：固定 COPY 吸收网络、紧凑坐标后端；
- algebraic_ttn/parity.py：列反射商、等值连接、外存 run；
- algebraic_ttn/symmetric.py：拓扑规划与行镜像子树；

- `algebraic_ttn/symmetric_parity.py` : 组合执行 DAG ;
- `src/merge_sorted_runs.cpp` : 精确 k 路有序归并 ;
- `python/contract_symmetric_parity_ttn.py` : 命令行入口 ;
- `REPORT_ALGEBRAIC_TTN.md` : 逐阶段实验日志与数据。

18. 最终理论判断

这条路线的本质不是“用神经网络猜出 N 皇后数”，也不是“把回溯换一个名字”。它是以下恒等式的工程化：

全局合法赋值数 = 局部 0/1 因子乘积的有限和 = 任意合法二叉次序下的精确张量收缩。

它的正确性来自：

- 局部约束分解；
- COPY 恒等式；
- 部分收缩不变量；
- 群作用下的轨道等价；
- 精确整数加法与乘法的结合律、交换律和分配律。

它的效率来自：

- 好的收缩树；
- 中间张量的实际稀疏性；
- 对称轨道的严格商表示；
- 同构子树的 DAG 复用；
- 外存排序归并。

它当前的限制来自：

- 完全图约束导致的线性宽度；
- 后段边界张量的指数包络；
- 共享键组产生的巨大连接基数；
- 对称压缩通常只有常数因子，且可能与稀疏性冲突。

因此最稳妥的结论是：

精确 TTN 收缩在理论和实现上都成立，但当前路线尚未把 N=15 压到本机可完成范围。

接下来真正值得攻克的不是“再把某个数组换一种容器”，而是同时降低收缩树的边界宽度、约束延迟和连接基数，并把不可避免的大连接改造成端到端的外存精确流水线。

精确 TTN 的 GPU 集群迁移路线

当前已实现的阶段

收缩器现在有两条精确路径。小连接保留 NumPy 参考实现；达到 CUDA 阈值的 流式连接执行

```

uint64 output_key / uint64 output_value
→ GPU 流式连接
→ GPU J_g = n_L(g)n_R(g)
→ GPU 流式连接
→ CUDA 流式连接 (i,j)
→ CUDA 流式连接 key/value
→ GPU sort/reduce-by-key
→ GPU run

```

因此主机不再为 CUDA 流式连接物化逐贡献的 left_rows/right_rows。每块的

```

uint64 output_key / uint64 output_value
→ GPU key
→ GPU key uint64
→ GPU key/value run

```

抽象成独立的 ExactBlockReducer。

提供两个实现：

- **numpy**：CPU 参考实现，也是默认后端；
- **cuda**：可选 CuPy 实现，在 CUDA GPU 上执行 key/value 排序和

reduce-by-key；传入多个设备时使用多 GPU 键范围分片。

此外已经实现：

- 预算预检产生的排序分组计划由流式路径直接复用，不再进行第二次排序；
- 八张卡各有一个在途组片，输入收集在对应 worker 线程内并行执行；
- H2D 输入和 D2H 归并结果使用 CUDA 锁页主存；
- GPU 内核融合组内配对、坐标解码、反射规范化和精确值生成；
- 小块自动回退 NumPy，大块才送入 GPU；
- 外部归并采用有限扇入的分层并行归并，不再一次打开数千个文件；
- 中间归并文件使用 partial 文件、原子改名和完成标记。

CUDA/CuPy 是可选依赖。CPU 环境不导入 CuPy，也不改变已有运行方式。请求 CUDA 但 CuPy 或设备不可用时，程序会在收缩开始前明确报错。

当前仍未完成的是：超出单卡排序显存时的多 GPU/外存共享键分组、持久化 join-shard manifest、进程重启后的缺失 shard 重算，以及跨节点调度。因此它 已经是单节点多 GPU 的贡献生成器，但还不能称为完整的多节点收缩器。

为什么不能直接调用 GEMM

稠密张量收缩可以通过指标重排写成矩阵乘法。但本路线故意不物化稠密中间 张量：例如 N=15 的 rank-9 包络已有

$$15^9=38\,443\,359\,375$$

个格点，仅一个 uint64 稠密数组约 307 GB；rank-13 包络更不可行。因此实际 算子不是常规稠密 GEMM，而是

共享坐标等值连接 → 稀疏贡献生成 → sort/reduce-by-key.

它更接近 GPU 数据库 join、radix sort、run-length encode 和 segmented reduction。再加上系数必须保持精确 uint64，V100 的 FP16/FP32 Tensor Core 不能直接代替整数收缩。GPU 仍然适合这条路线，但需要面向稀疏连接设计 kernel，而不是把不可承受的零元素补回去。

使用

CPU 参考后端：

```
python3 python/contract_symmetric_parity_ttn.py \
  --n-min 1 --n-max 11 \
  --join-chunk-pairs 1000000 \
  --block-reducer numpy
```

CUDA 节点上安装与集群 CUDA 运行时匹配的 CuPy 后：

```
CUDA_VISIBLE_DEVICES=0 \
python3 python/contract_symmetric_parity_ttn.py \
  --n-min 11 --n-max 11 \
  --join-chunk-pairs 4000000 \
  --streaming-merge-strategy sorted-runs \
  --block-reducer cuda \
  --cuda-device 0
```

单机四卡：

```
CUDA_VISIBLE_DEVICES=0,1,2,3 \
python3 python/contract_symmetric_parity_ttn.py \
  --n-min 11 --n-max 11 \
  --join-chunk-pairs 8000000 \
  --streaming-merge-strategy sorted-runs \
  --block-reducer cuda \
  --cuda-devices 0,1,2,3 \
  --cuda-min-records 1000000 \
  --cuda-records-per-device 2000000
```

--cuda-device 和 --cuda-devices 都使用 CUDA_VISIBLE_DEVICES 过滤后的 进程内编号；设备列表会覆盖单设备参数。输出 summary 新增：

- block_reducer_backend;
- block_reducer_device;
- block_reducer_devices;
- block_reducer_calls;
- block_reducer_input_records;
- block_reducer_output_records;
- block_reducer_seconds;
- block_reducer_async_submissions;
- block_reducer_cpu_fallback_calls;
- block_reducer_cpu_fallback_records;
- block_reducer_gpu_device_dispatches.

--cuda-min-records 以下的块走 NumPy 精确归并；--cuda-records-per-device 控制一个块按贡献数最多激活多少张卡。二者只改变 执行位置，不改变连接记录或算术。

为什么仍然精确

GPU 后端只处理 uint64 键和值：

1. 按键重排记录；
2. 对相同键的值做整数加法；
3. 返回相同的严格递增唯一键。

它不使用浮点数，不丢弃记录，也不改变收缩树和对称轨道规则。在既有 $N^2 \leq 2^{64}$ 安全界内，整数加法重结合不改变结果。CPU 参考后端保留为逐节点和端到端回归基准。

当前多 GPU 算法

旧的内存内小连接仍可按输出键范围分给多张卡。新的流式主路径改为先按共享键组的精确连接量做全局贡献编号，再切成近似等长的工作片。每个工作片保留组起点、左右计数和组内起始偏移，不携带逐贡献行号。于是：

- 八张卡按贡献数而不是输出键跨度取得工作；
- 巨大单组也能从组内偏移处分块；
- 每张卡独立生成和局部 reduce-by-key；
- 不同卡可能产生相同输出键，所以有序 run 最后做精确外部归并。

源张量当前仍在主机或 mmap 文件中；worker 只收集自己组片覆盖的连续排序区间，再通过锁页内存传输。后续需要把常用输入 shard 驻留 GPU/NVMe 并加入 NUMA 绑定，才能进一步减少 PCIe 和主机 gather 成本。

8×V100 实测

服务器为 8 张 Tesla V100-SXM2 32 GB。所有结果使用相同的精确张量网络、uint64 算术和拓扑计划。

实验	旧实现	融合 CPU	自适应 8 GPU
N=11 完整收缩	约 19.4 s（剖析运行）	14.34 s	5.34 s
N=11 块归并	2.50–3.52 s	2.57 s	1.04 s
N=12 完整收缩	—	—	29.64 s
N=15 同预算部分收缩	149.90 s	—	95.16 s
N=15 块归并	19.74 s	—	12.97 s

新 GPU 贡献内核的 N=11 运行得到精确标量 2680；N=12 得到 14200，并处理最大 259,757,335 项的精确连接。N=12 使用 8M 贡献片、GPU 共享坐标编码、锁页传输和 128 路直接归并。旧 N=15 两次运行都完成 111/120 个计划节点，在同一个 1,150,580,209-pair 下一连接前按 332,706,850 的预算停止；所以表中比较的是同一精确部分前沿，不是近似答案。N=15 进程峰值 RSS 从约 8.79 GB 变为约 8.76 GB，主要收益来自减少重复坐标读取、流水重叠和避免小块过度多卡调度。

单 GPU 数据通路

当前 CUDA 流式路径已经去掉 CPU 生成的 left_rows/right_rows。对每个共享键组 g，计算

$$J_g = n_L(g) n_R(g)$$

以及组前缀和。GPU 线程根据贡献编号直接恢复

$$i = \lfloor p / n_R(g) \rfloor, j = p \bmod n_R(g),$$

并直接生成输出键和值。目标流水线是：

```

GPU shared-key encode
→ radix sort
→ run-length encode
→ □□□ merge/intersection
→ 64-bit group prefix scan
→ fused pair contribution kernel
→ radix sort output key/value
→ reduce-by-key
→ host/SSD sorted run

```

这样每个连接不再额外物化两个 uint32 行号，可少用至少 8 字节临时空间。

完整多 GPU / 多节点 join 分解

单个收缩节点内部可按共享键组分片。协调进程先构造只含拓扑和组偏移的精确 manifest，再按预计连接量平衡分片：

```

contraction node
├── shard 0: groups [g0, g1) → sorted run 0
├── shard 1: groups [g1, g2) → sorted run 1
├── ...
└── shard m: groups [gm, end) → sorted run m

```

每个 GPU worker：

1. 只读取自己的左右组区间；
2. 生成全部匹配贡献；
3. 在本地 GPU 上排序和 reduce-by-key；
4. 写出带校验元数据的严格递增 run。

协调器等待所有 shard 完成，然后使用现有 C++ k 路归并器精确合并。不同 shard 可能产生相同输出键，所以最终全局归并不可省略。

分布式执行不把不同收缩树节点任意并行：父节点必须等待两个子张量完成。主要并行度来自单个超大 join 内的共享键组分片，以及收缩 DAG 中确实彼此独立的节点。

集群存储要求

GPU 集群版仍需要共享或分布式外存。中间张量采用两个顺序文件：

```

keys.bin    little-endian uint64, strictly increasing
values.bin  little-endian uint64, same length

```

worker run 可写入节点本地 NVMe，随后分层归并：

1. GPU 内块归并；
2. 节点本地 run 归并；
3. 机架或作业级全局归并。

应避免让所有 GPU 同时随机访问一个共享网络文件。大张量输入宜按组索引和连续行区间分片，并对热输入做节点本地缓存。

后续实施顺序

1. 已完成：可选单 GPU CUDA 块归并后端；
2. 已完成：单进程多 GPU 键范围块归并；
3. 已完成：CUDA 与 NumPy 在 8×V100 上做 N=11 端到端一致性测试；

4. 已完成：CPU/GPU 流水重叠和按块大小自适应设备调度；
5. 已完成：GPU fused pair contribution kernel，消除左右行号数组；
6. 部分完成：单卡可容纳时的 GPU shared-key sort 和连接预检；
7. 生成可重启、可校验的 join-shard manifest；
8. 已完成：单节点多 GPU join worker；
9. 部分完成：有限扇入的单节点并行 run 分层归并；
10. 检查点、失败 shard 重试和最终标量证书。

只有第 7 步以及跨节点 worker 完成后，程序才是真正的集群 join 分片版本。当前已经支持单节点多 GPU 完整生成贡献，但不应宣传为跨机器分布式收缩。

V100 与 A100 的部署建议

- V100 16/32 GB：从每个全局块 1-4 百万条贡献开始测量；
- A100 40/80 GB：从 4-16 百万条开始测量；
- 块大小是全部 GPU 合计的输入块，不是每卡各自再复制一份；
- 不要在同一等范围池中混用 V100 与 A100，否则较慢设备可能成为同步点；
- 每个节点优先使用本地 NVMe 写 run，再做分层外存归并；
- 第一轮基准必须同时记录 GPU 利用率、PCIe/NVLink 传输、块归并时间和磁盘

写入时间。

这些范围只是安全起始值，最终应根据实际显存、CuPy 排序工作区和键分布调整。

附录：复现入口与验证边界

当前推荐入口：python/contract_symmetric_parity_ttn.py

```
CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7 python3 \  
python/contract_symmetric_parity_ttn.py \  
--n-min 12 --n-max 12 --join-chunk-pairs 8000000 \  
--block-reducer cuda --cuda-devices 0,1,2,3,4,5,6,7
```

8×V100 路线已经精确得到 N=11 标量 2680 和 N=12 标量 14200。N=15 数据是同一预算下完成 111/120 概念节点后的精确部分前沿，并未得到最终标量。A100 尚需按目标服务器的显存、CUDA/CuPy 版本和本地 NVMe 条件重新基准测试。